

CS6650 Final

Sai Karthikeyan Sura
April 12th, 2026

Best Score: 181/190 (110 correctness, 71 load) **Total Submissions:** ~35

1. How many submissions before passing all critical scenarios, and most common failure?

Approximately 3 submissions were needed to pass all critical scenarios (S1 through S5). The most common early failure was S3 (Async Photo Upload), where the photo remained in `processing` status past the 30 second timeout because the SQS worker was not completing the DynamoDB status update fast enough. Adding optimistic inline completion in the API handler (bypassing SQS for the happy path) resolved this.

2. Where are photo files stored, and why?

Photos are stored in Amazon S3 with a public read bucket policy. S3 was chosen because the contract requires the photo URL to return HTTP 200 when fetched directly by the ChaosArena grader. S3 provides durable, publicly accessible object storage without needing to serve files through the application layer, which would consume ECS task bandwidth. Transfer Acceleration is enabled for faster upload throughput.

3. Deployment setup.

The system runs on AWS ECS Fargate in us-west-2 with 6 API tasks (2048 CPU, 4096 MB each) behind an Application Load Balancer, and 12 worker tasks (512 CPU, 1024 MB each) consuming from SQS. DynamoDB (on-demand) stores three tables. S3 stores photo blobs. SQS Standard Queue with a Dead Letter Queue handles async job delivery. All ECS tasks run in private subnets with VPC Gateway Endpoints for S3 and DynamoDB to bypass NAT gateway latency.

4. Reverse proxy or load balancer?

An AWS Application Load Balancer distributes incoming HTTP requests across the 6 API Fargate tasks using round-robin. It performs health checks on `/health` every 15 seconds and deregisters unhealthy targets within 10 seconds. The ALB also provides the single public DNS endpoint required by the ChaosArena submission.

5. How does the background worker get notified?

The primary path is optimistic inline completion: the API handler uploads to S3 and marks the photo as `completed` directly, without involving the worker. If inline completion fails, the handler enqueues a job to SQS Standard Queue as a fallback. Workers long-poll SQS with 20-second wait times and 120-second visibility timeout. The DLQ catches messages that fail 3 times.

6. Why must `seq` be assigned in the POST handler, and how is correctness ensured?

Assigning `seq` in the handler guarantees the client receives a deterministic, monotonically increasing sequence number in the 202 response, before any async processing begins. If assigned in the worker, concurrent uploads could complete out of order and produce non-monotonic sequences. Correctness under concurrency is ensured via DynamoDB `UpdateItem` with `ADD` semantics on an atomic counter table (`album_counters`), which guarantees unique, monotonically increasing values even under parallel requests to the same album.

7. What happens if the worker crashes mid-processing?

The SQS message remains invisible for 120 seconds (visibility timeout), then reappears for another worker to retry. After 3 failed attempts (`maxReceiveCount=3`), the message moves to the Dead Letter Queue. The photo record stays in `processing` status. The worker also checks

idempotency: if the photo was already completed or deleted, it simply acknowledges the message without reprocessing.

8. Database schema.

Three DynamoDB tables, all on-demand billing:

- `albums` (PK: `album_id`) stores album metadata (title, description, owner).
- `album_counters` (PK: `album_id`) stores the atomic `next_seq` counter per album.
- `photos` (PK: `album_id`, SK: `photo_id`) stores photo records with status, seq, s3_key, and url fields.

Separating the counter table avoids hot-partition contention on the photos table during concurrent uploads to the same album.

9. Indexes?

No secondary indexes were added. DynamoDB on-demand mode handles all access patterns with the primary key alone. `albums` uses single-item `GetItem` by `album_id`. `photos` uses composite key (`album_id`, `photo_id`) for point reads. `ListAlbums` uses a full `Scan`, which is acceptable because the table is small and the scenario (S5/S13) tolerates scan latency. Adding a GSI would not improve the bottleneck scenarios.

10. Hardest load testing scenario and bottleneck.

S12 (Concurrent Photo Uploads) was consistently the hardest, scoring 5 to 8 out of 15 across all runs. The bottleneck is S3 `PutObject` latency under concurrent load: p95 ranged from 3900ms to 5700ms. Multiple approaches were tested including Fargate vs EC2 (c5.xlarge with dedicated 10 Gbps), goroutine-based background uploads, direct `PutObject` bypassing the upload manager, S3 Transfer Acceleration, connection pool tuning, and hash-prefixed S3 keys. None reduced p95 below ~3900ms, suggesting the bottleneck is S3 service-side request scheduling rather than client-side throughput.

11. Single most impactful change.

Switching from synchronous (`ParseMultipartForm` → S3 upload → DynamoDB → SQS → 202) to the early-flush streaming pattern: parse multipart headers, allocate seq, create DynamoDB record, flush 202 via `http.Flusher`, then stream directly to S3 in the same handler. This eliminated memory buffering, returned 202 in under 50ms, and improved the overall score from 161 to 181 (a 20-point gain).

12. Handling concurrent writes.

Album creates use conditional `PutItem` with `attribute_not_exists(album_id)` for idempotency, falling back to unconditional `PutItem` on `ConditionalCheckFailedException`. Photo sequence allocation uses DynamoDB atomic `UpdateItem` with `ADD` on the counters table. Photo completion uses conditional `UpdateItem` with `attribute_exists(album_id)` to prevent zombie records if a delete races with completion. Parallel deletes are handled by issuing S3 and DynamoDB deletes concurrently via goroutines with a `sync.WaitGroup`.

13. Specific bug diagnosed via ChaosArena event logs.

S9 (Delete Before Complete) failed with “context cancelled during orphan poll” when using the goroutine-based handler. The event log showed the grader uploading 5 photos, deleting all 5, then polling the S3 URLs. The grader found HTTP 200 at an S3 URL that should have been deleted.

Root cause: the goroutine uploaded to S3 after the handler returned 202, then `CompletePhoto` failed with `ConditionalCheckFailedException` (record already deleted), but the S3 object was never cleaned up, leaving an orphan. Fix: added `store.IsConditionalCheckFailed()` check in the goroutine to delete the orphan S3 object on completion failure.

14. Local testing before ChaosArena.

A Locust load test suite (`locust/`) was written to approximate S11 through S15 scenarios with configurable concurrency. A shell-based smoke test script (`scripts/smoke_test.sh`) validated all 7 endpoints against the deployed ALB before every ChaosArena submission. The deploy script (`scripts/deploy.sh`) runs terraform apply, Docker build/push, ECS force-new-deployment with task stabilization wait, and the full smoke test as a single pipeline.

15. What would change with another week?

Investigate using S3 Express One Zone (directory buckets) in us-west-2. S3 Express provides single-digit-millisecond PutObject latency compared to the 50 to 200ms baseline of standard S3, which would directly address the S12 bottleneck. The trade-off is single-AZ durability and higher per-request cost, but for a graded assignment where latency determines the score, it would likely push S12 from 8/15 to 15/15 and reach 190.

16. How did you add value over Claude?

Claude served as a code generation accelerator, producing initial file scaffolds (Go handler stubs, Terraform module skeletons, Dockerfile templates) from high-level architectural specifications. All system design decisions were made independently: choosing DynamoDB over PostgreSQL for single-digit-millisecond point reads, designing the three-table schema with a separate atomic counter table to avoid hot partitions, selecting ECS Fargate over Lambda for persistent connection pooling, and architecting the optimistic inline completion pattern with SQS as a durable fallback. The performance optimization cycle was driven by systematic analysis of ChaosArena scoring data across 35+ submissions: identifying that S12 (concurrent photo uploads) was the consistent bottleneck, hypothesizing that S3 PutObject tail latency under concurrency was the root cause rather than application-level overhead, and methodically testing six handler architectures (synchronous, early flush streaming, goroutine with background upload, hybrid size-based split, direct PutObject bypass, goroutine with S9 orphan cleanup) to validate that hypothesis. Infrastructure-level experimentation included migrating from Fargate to EC2 c5.xlarge instances to test whether dedicated 10 Gbps network bandwidth would reduce S3 latency (it did not, confirming the bottleneck was S3 service-side), tuning connection pool sizes, and enabling S3 Transfer Acceleration. Debugging required correlating Terraform state, ECS service events, DynamoDB table accumulation patterns, and ChaosArena event logs to diagnose issues like incorrect handler versions deployed via git checkout, ENI limits blocking EC2 task scheduling, and orphan S3 objects causing S9 correctness failures. Claude accelerated the implementation of each iteration, but the experimental methodology, architectural reasoning, and operational debugging that drove the score from 161 to 181 were the core contributions.